

SQL practice Exercise

Exercise 4

Q1 SQL JOINS

```
SELECT e.Employee ID,  
       e.first_name,  
       e.last_name,  
       e.Department,  
       e.Salary,  
       P.Project-ID,  
       P.Project-name,  
       P.Budget,  
       P.Status
```

```
FROM Employee e  
INNER JOIN Projects P ON e.Employee ID = P.Employee ID;
```

Outcome

Employee ID	first_name	Last-name	Department	Salary	ProjectID	Project-name
1	John	Doe	IT	70000	101	Development
1	John	Doe	IT	70000	103	Audit
2	Alice	Smith	HR	60000	102	Employee Training
3	Bob	Johnson	Finance	75000	104	Financial Analysis
5	Emma	Wilson	Sales	65000	105	Market

Q2.

```
SELECT e.employee ID,
       e. first-name,
       e. last-name,
       e. Department,
       e. Salary,
       p. Project ID,
       p. Project-name,
       p. Budget
```

FROM Employees e

LEFT JOIN Project p ON e.employee ID = p.employee ID ;

Outcome

Employee ID	first-name	last-name	Department	Salary	Project ID	Project-name
1	John	Doe	IT	70000	101	AI Development
1	John	Doe	IT	70000	103	Cyber Security Audit
2	Alice	Smith	HR	60000	102	Employee Training
3	Bob	Johnson	Finance	75000	104	Financial Analysis
4	David	Brown	IT	72000	Null	Null
5	Emma	Wilson	Sales	65000	105	Market Expansion

Q3

```
SELECT p.Project ID,
       P. Project_name,
       P. Budget
       P. Status
       e. Employee ID,
       e. first-name,
       e. last-name,
       e. Department,
       e. Salary
```

FROM Employee.e

RIGHT JOIN Projects P ON e.EmployeeID = P.Employee ID;

Outcome

Project ID	Project_name	Budget	Status	Employee ID	First name	Last name
101	AI Development	100000	Completed	1	John	Doe
102	Employee Training	50000	Ongoing	2	Alice	Smith
103	Cybersecurity Audit	75000	Pending	1	John	Doe
104	Financial Analysis	90000	Ongoing	3	Bob	Johnson
105	Market Expansion	65000	Completed	5	Emma	Wilson

Q4

```
SELECT e.EmployeeID,  
       e.Firstname,  
       e.Lastname,  
       e.Department,  
       e.Salary,  
       P.ProjectID,  
       P.Projectname,  
       P.Budget,  
       P.Status
```

```
FROM Employee e
```

```
Full OUTER JOIN Projects p ON e.EmployeeID = p.EmployeeID;
```

3. Filtering Statement

```
SELECT EmployeeID,  
        Firstname,  
        lastname,  
        Department,  
        Salary  
FROM Employees  
WHERE Salary > 70000;
```

Outcome

EmployeeID	Firstname	lastname	Department	Salary
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000
6	Michael	Clerk	Finance	80000

8. **SELECT** EmployeeID,
 Firstname,
 Lastname,
 Department
~~FROM~~ Salary
~~FROM~~ Employees
~~WHERE~~ Department IN ('IT', 'Finance');

~~EmployeeID~~

Outcome

EmployeeID	Firstname	Lastname	Department	Salary
1	John	Doe	IT	70000
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000
6	Michael	Clerk	Finance	80000

9. **SELECT** Project-ID,
Projectname,
Budget,
Status
FROM Projects
WHERE Status $\neq$ 'Completed'

Outcome

Project ID	Projectname	Budget	Status
102	Employee training	50000	Ongoing
103	Cybersecurity Audit	75000	Pending
1021	Financial Analysis	90000	Ongoing
106	Risk Management	80000	Pending

10. SELECT Project ID,
Project name
Budget
Status

FROM Projects

WHERE Budget > 70000 AND status != 'Completed'

Outcome

Project ID	Project Name	Budget	Status
103	Cyber security Audit	75000	Pending
104	Financial Analysis	90000	On going
106	Risk management	80000	Pending


```

11. SELECT Employee ID,
        Firstname,
        Lastname
        Department
        Salary
        City
FROM employees
WHERE city IN ('New York', 'Toronto')
ORDER BY Salary DESC

```

Outcome

EmployeeID	Firstname	Lastname	Department	Salary	City
6	Michael	Clert	Finance	80000	New York
3	Bob	Johnson	Finance	75000	Toronto
1	John	Doe	IT	70000	New York

D. **SELECT** EmployeeID,
 firstname,
 lastname,
 Department,
 Salary
FROM employee
WHERE Salary DESC
LIMIT 3

Outcome

EmployeeID	firstname	lastname	Department	Salary
6	Michael	Clerk	Finance	80000
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000

14. Aggregate functions with GROUP BY & HAVING

13.

```
SELECT Department,  
       Sum(Salary) AS total-Salary  
FROM   Employees  
GROUP BY Department  
ORDER BY total-Salary DESC
```

Outcome

Department	total-Salary
Finance	185 000
IT	142 000
HR	60 000
Sales	65 000

14. **SELECT** City,
 AVG(salary) AS Average_Salary
FROM Employees
GROUP BY City
HAVING AVG(salary) > 65000
ORDER BY Average_Salary DESC;

Outcome

City	Average - Salary
New York	76000
Toronto	75000
London	72000

5. **SELECT** Department,
COUNT (Employee ID) AS Employee Count
FROM Employees
GROUP BY Department
HAVING COUNT (Employee ID) > 1
ORDER BY Employee Count DESC;

Outcome

Department	Employee Count
IT	2
Finance	2

16. **SELECT** Status,
Count(project ID) AS projectCount
FROM Projects
GROUP BY Status
HAVING Count(Project ID) ≥ 2
ORDER BY ProjectCount DESC;

Status	projectCount
Pending	2
Ongoing	2

```

F. SELECT e.EmployeeID,
           e.firstname,
           e.lastname,
           Sum(p.Budget) AS total-Project-Budget
FROM       Employee.e
INNER JOIN Project.p ON e.EmployeeID = p.EmployeeID
GROUP BY   e.employeeID, e.firstname, e.lastname
HAVING     Sum(p.Budget) > 150000
ORDER BY   TotalProjectBudget DESC ;

```

Outcome

EmployeeID	firstname	lastname	total-Project-Budget
1	John	Doe	175000